Case Study 9: Student Marks Matrix Analysis

Python Activity-Based Learning Assignment

1. Problem Statement

Develop a Python program to analyze students' marks across multiple subjects using a nested list (matrix). The program should calculate each student's total and average marks, identify the highest-scoring subject, and classify students according to their average marks.

2. Objectives

- To understand lists and nested lists in Python.
- To store marks in a matrix format.
- To use loops to process rows and columns.
- To calculate totals and averages using arithmetic operators.
- To identify the highest-scoring subject.
- To classify students using conditional statements.

3. Assumptions

1. The program analyzes three students and three subjects.
2. Each mark is between 0 and 100.
3. Each student has marks for Mathematics, Python, and English.
4. A student's total is the sum of the three subject marks.
5. The average is calculated by dividing the total by three.
6. Classification is based on average marks:
    - Average 75 or above: Excellent
    - Average 60 to below 75: Good
    - Average 40 to below 60: Average
    - Average below 40: Needs Improvement

These are example assumptions for this implementation.

4. Algorithm

1. Start the program.

2. Define the subject names.
3. Create an empty nested list to store students' marks.
4. Enter marks for each student in all three subjects.
5. Validate that every mark is between 0 and 100.
6. Calculate the total and average marks for each student.
7. Classify each student according to their average.
8. Calculate the total marks obtained in each subject.
9. Identify the subject with the highest total marks.
10. Display the complete analysis report.
11. Stop the program.

5. Flowchart

Start
Define students and subjects
Enter marks in a nested list
Calculate totals and averages
Classify each student
Calculate subject totals and find the highest
Display the analysis report
Stop

6. Pseudocode

```
None
START

subjects = Mathematics, Python, English
CREATE empty nested list marks

FOR each student:
    INPUT marks for all subjects
    VALIDATE marks between 0 and 100
    STORE marks in nested list

FOR each student:
    total = sum of subject marks
    average = total / number of subjects

    IF average >= 75:
```

```
        classification = "Excellent"
    ELSE IF average >= 60:
        classification = "Good"
    ELSE IF average >= 40:
        classification = "Average"
    ELSE:
        classification = "Needs Improvement"

    DISPLAY total, average, classification

FOR each subject:
    CALCULATE total marks across students

IDENTIFY subject with highest total
DISPLAY highest-scoring subject

STOP
```

7. Python Source Code

Save this program as `marks_matrix.py`.

None

```
print("===== STUDENT MARKS MATRIX ANALYSIS =====")

students = ["Student 1", "Student 2", "Student 3"]
subjects = ["Mathematics", "Python", "English"]

marks = []

# Input marks for each student
for student in students:
    print("\nEnter marks for", student)
    student_marks = []
```

```
    for subject in subjects:
        while True:
            try:
                mark = float(
                    input(subject + " marks (0-100): ")
                )

                if 0 <= mark <= 100:
                    student_marks.append(mark)
                    break
                else:
                    print("Marks must be between 0 and 100.")

            except ValueError:
                print("Please enter a valid number.")

    marks.append(student_marks)

# Analyze each student's performance
print("\n===== STUDENT PERFORMANCE REPORT =====")

for i in range(len(students)):
    total = sum(marks[i])
    average = total / len(subjects)

    if average >= 75:
        result = "Excellent"
    elif average >= 60:
        result = "Good"
    elif average >= 40:
        result = "Average"
    else:
        result = "Needs Improvement"

    print("\nStudent:", students[i])
```

```
    print("Marks:", marks[i])
    print("Total:", total)
    print("Average:", round(average, 2))
    print("Classification:", result)

# Calculate total marks for each subject
subject_totals = []

print("\n===== SUBJECT-WISE TOTALS =====")

for j in range(len(subjects)):
    total = 0

    for i in range(len(students)):
        total += marks[i][j]

    subject_totals.append(total)
    print(subjects[j], ":", total)

# Find the highest-scoring subject
highest_total = max(subject_totals)
highest_subjects = []

for j in range(len(subjects)):
    if subject_totals[j] == highest_total:
        highest_subjects.append(subjects[j])

print("\nHighest-scoring subject(s):",
      ", ".join(highest_subjects))
print("Highest subject total:", highest_total)

print("\nAnalysis completed.")
```

8. Test Cases

Student	Mathematics	Python	English	Total	Average	Classification
Student 1	80	75	90	245	81.67	Excellent
Student 2	65	85	70	220	73.33	Good
Student 3	90	95	80	265	88.33	Excellent

Subject-wise totals:

Subject	Total marks
Mathematics	235
Python	255
English	240

Expected result: Python is the highest-scoring subject, with a total of 255 marks.

Sample output

The following output corresponds to the test marks in the table above.

None

```
===== STUDENT MARKS MATRIX ANALYSIS =====

===== STUDENT PERFORMANCE REPORT =====

Student: Student 1
Marks: [80.0, 75.0, 90.0]
Total: 245.0
Average: 81.67
Classification: Excellent

Student: Student 2
Marks: [65.0, 85.0, 70.0]
Total: 220.0
Average: 73.33
Classification: Good

Student: Student 3
Marks: [90.0, 95.0, 80.0]
Total: 265.0
Average: 88.33
Classification: Excellent

===== SUBJECT-WISE TOTALS =====
Mathematics : 235.0
Python : 255.0
English : 240.0

Highest-scoring subject(s): Python
Highest subject total: 255.0

Analysis completed.
```

9. Output Screenshots

```
print("===== STUDENT MARKS MATRIX ANALYSIS =====")

students = ["Student 1", "Student 2", "Student 3"]
subjects = ["Mathematics", "Python", "English"]

marks = []

# Input marks for each student
for student in students:
    print("\nEnter marks for", student)
    student_marks = []

    for subject in subjects:
        while True:
            try:
                mark = float(
                    input(subject + " marks (0-100): ")
                )

                if 0 <= mark <= 100:
                    student_marks.append(mark)
                    break
                else:
                    print("Marks must be between 0 and 100.")

            except ValueError:
                print("Please enter a valid number.")

    marks.append(student_marks)

# Analyze each student's performance
print("\n===== STUDENT PERFORMANCE REPORT =====")

for i in range(len(students)):
    total = sum(marks[i])
    average = total / len(subjects)

    if average >= 75:
        result = "Excellent"
    elif average >= 60:
        result = "Good"
    elif average >= 40:
        result = "Average"
    else:
        result = "Needs Improvement"

    print("\nStudent:", students[i])
    print("Marks:", marks[i])
```

```
# Calculate total marks for each subject
subject_totals = []

print("\n===== SUBJECT-WISE TOTALS =====")

for j in range(len(subjects)):
    total = 0

    for i in range(len(students)):
        total += marks[i][j]

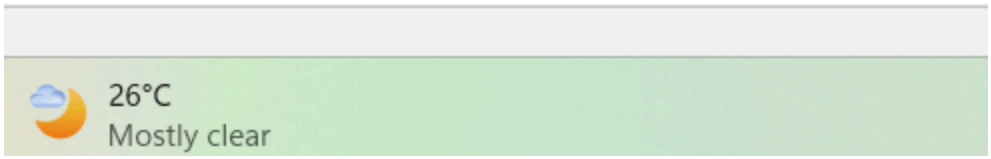
    subject_totals.append(total)
    print(subjects[j], ":", total)

# Find the highest-scoring subject
highest_total = max(subject_totals)
highest_subjects = []

for j in range(len(subjects)):
    if subject_totals[j] == highest_total:
        highest_subjects.append(subjects[j])

print("\nHighest-scoring subject(s):",
      ", ".join(highest_subjects))
print("Highest subject total:", highest_total)

print("\nAnalysis completed.")
```



```
===== STUDENT MARKS MATRIX ANALYSIS =====

Enter marks for Student 1
Mathematics marks (0-100): 80
Python marks (0-100): 75
English marks (0-100): 90

Enter marks for Student 2
Mathematics marks (0-100): 65
Python marks (0-100): 85
English marks (0-100): 70

Enter marks for Student 3
Mathematics marks (0-100): 90
Python marks (0-100): 95
English marks (0-100): 80

===== STUDENT PERFORMANCE REPORT =====

Student: Student 1
Marks: [80.0, 75.0, 90.0]
Total: 245.0
Average: 81.67
Classification: Excellent

Student: Student 2
Marks: [65.0, 85.0, 70.0]
Total: 220.0
Average: 73.33
Classification: Good

Student: Student 3
Marks: [90.0, 95.0, 80.0]
Total: 265.0
Average: 88.33
Classification: Excellent

===== SUBJECT-WISE TOTALS =====
Mathematics : 235.0
Python : 255.0
English : 240.0

Highest-scoring subject(s): Python
Highest subject total: 255.0

Analysis completed.
```

10. Conclusion

The Student Marks Matrix Analysis program demonstrates how Python nested lists can store and process marks for multiple students and subjects. It uses loops, conditional statements, arithmetic operations, and list functions to calculate totals, averages, student classifications, and the highest-scoring subject